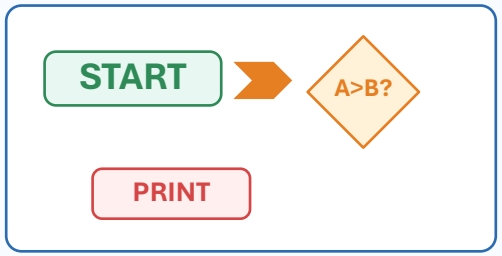
BASICS OF PROGRAMMING



বিষয়সমূহ: Programming, Language Classification, Translator, Algorithm, Flowchart, Program Planning

Problem Solving | Logic | Steps | Code

১. Programming এবং Program

Computer Programming-এর মৌলিক ধারণা

Computer Programming কী?

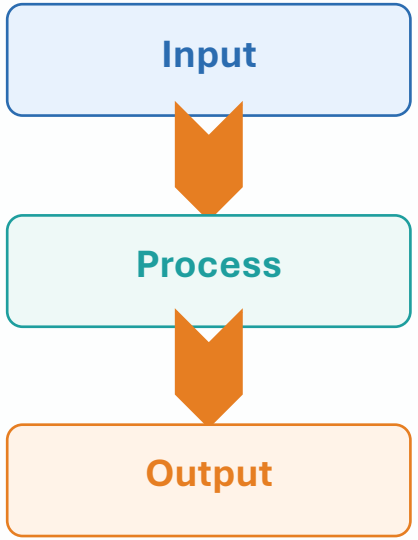
কম্পিউটারকে কোনো নির্দিষ্ট কাজ করানোর জন্য ধাপে ধাপে নির্দেশনা লেখা ও সাজানোর প্রক্রিয়াই Programming।

Program কী?

কম্পিউটারকে কাজ করানোর জন্য লিখিত নির্দেশনার সমষ্টিকে Program বলে।

Programming কেন দরকার?

- সমস্যা সমাধান করতে
- গাণিতিক হিসাব ও ডেটা প্রক্রিয়াকরণ করতে
- সফটওয়্যার, অ্যাপ ও ওয়েবসাইট তৈরি করতে
- পুনরাবৃত্ত কাজ স্বয়ংক্রিয় করতে
- দ্রুত, নির্ভুল এবং সংগঠিত ফলাফল পেতে



সহজ উদাহরণ: ২টি সংখ্যা ইনপুট নাও → যোগ করো → ফলাফল দেখাও

২. Programming Language এবং Classification

Programming Language-এর ধরন ও বৈশিষ্ট্য

Programming Language হলো এমন একটি ভাষা যার মাধ্যমে মানুষের নির্দেশ কম্পিউটারকে বোঝানো হয়।

Machine Language

০ ও ১ দিয়ে লেখা হয়
- কম্পিউটার সরাসরি বুঝে
- খুব দ্রুত
- মানুষের জন্য কঠিন

Assembly Language

Mnemonic code ব্যবহার হয়
- যেমন ADD, SUB, MOV
- Assembler লাগে
- Machine dependent

High Level Language

মানুষের ভাষার কাছাকাছি
- যেমন C, Python, Java
- লেখা সহজ
- Compiler/Interpreter লাগে

সবচেয়ে কঠিন — মাঝামাঝি — সবচেয়ে সহজ

মনে রাখো: Language যত High-Level, মানুষের জন্য তত সহজ; আর Machine Language কম্পিউটারের জন্য সবচেয়ে স্বাভাবিক।

৩. Language Comparison

Machine, Assembly ও High Level Language-এর তুলনা

বিষয়	Machine	Assembly	High Level
লেখার ধরন	0 ও 1	Mnemonic code	English-like syntax
বোঝা	খুব কঠিন	তুলনামূলক সহজ	সবচেয়ে সহজ
গতি	খুব দ্রুত	দ্রুত	তুলনামূলক ধীর
নির্ভরতা	Machine dependent	Machine dependent	Machine independent
অনুবাদক	লাগে না	Assembler	Compiler / Interpreter
উদাহরণ	Binary instruction	ADD, MOV, SUB	C, C++, Java, Python

Exam tip: Machine ও Assembly হলো low-level; High Level Language সাধারণত application development-এর জন্য সবচেয়ে বেশি ব্যবহৃত।

8. Translator Programs

১.৩

Assembler, Compiler এবং Interpreter

কম্পিউটার সরাসরি High Level Language বোঝে না; তাই Source Program-কে Machine Language-এ রূপান্তর করতে Translator Program লাগে।

Source Program → **Translator Program** → **Machine Code**

Assembler

Assembly Language-কে Machine Language-এ রূপান্তর করে।
উদাহরণ: ADD, MOV, SUB নির্দেশ অনুবাদ।

Compiler

পুরো প্রোগ্রাম একসাথে অনুবাদ করে।
Error list একসাথে দেখায়।
Execution তুলনামূলক দ্রুত।

Interpreter

এক লাইন করে অনুবাদ ও execute করে।
যে লাইনে error, সেখানে থেমে যায়।
Debugging সহজ।

৫. Compiler vs Interpreter

দুই ধরনের Translator-এর পার্থক্য

Compiler

- পুরো Program একবারে translate করে
- সব error একসাথে দেখায়
- Object code তৈরি করতে পারে
- Compile শেষে run দ্রুত হয়
- বড় প্রোগ্রামের জন্য উপযোগী

Interpreter

- Line by line translate ও execute করে
- Error হলে সেই লাইনেই থামে
- Debugging সহজ
- প্রতিবার run-এ translation লাগে
- ছোট Program test করার জন্য ভালো

উদাহরণ: C/C++ → Compiler | Python → Interpreter
(ব্যবহারিক ধারণা)

৬. Algorithm এবং Flowchart

Step-by-step সমাধান ও তার চিত্রভিত্তিক উপস্থাপন

Algorithm

কোনো সমস্যা সমাধানের জন্য ধাপে ধাপে সুশৃঙ্খল নির্দেশনার সমষ্টিকে Algorithm বলে।

ভালো Algorithm-এর বৈশিষ্ট্য:

- পরিষ্কার ধাপ
- যৌক্তিক ক্রম
- সীমিত সংখ্যা step
- নির্দিষ্ট ফলাফল

Flowchart

Algorithm-কে বিভিন্ন symbol ব্যবহার করে চিত্র আকারে প্রকাশ করাকে Flowchart বলে।

উপকারিতা:

- ধাপ সহজে বোঝা যায়
- ভুল শনাক্ত করা সহজ
- Program design স্পষ্ট হয়
- অন্যকে বুঝানো সহজ

Algorithm → Flowchart → Code

৭. Flowchart Symbols-এর ব্যবহার

১.৫

প্রতিটি Symbol-এর কাজ আলাদা

চিহ্ন	নাম	ব্যবহার
	Terminal	Program কোথা থেকে শুরু এবং কোথায় শেষ হবে তা বোঝায়।
	Process	হিসাব, assignment বা processing ধাপ বোঝায়।
	Input / Output	ডেটা input নেওয়া বা output দেখানো হয়।
	Decision	শর্ত যাচাই করে Yes/No branch তৈরি করে।
	Flow line / Connector	ধাপের দিক ও সংযোগ দেখায়।

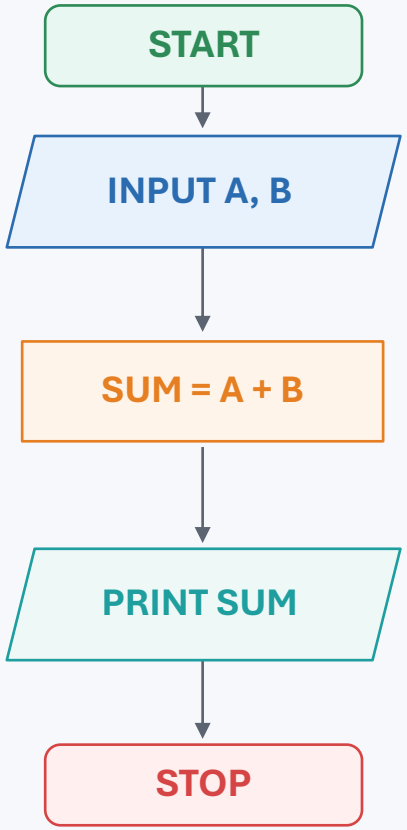
৮. Simple Problem Example: যোগফল নির্ণয়

Algorithm এবং Flowchart—দুইভাবেই সমাধান

Algorithm: দুটি সংখ্যা যোগ

১. Start
২. Input A, B
৩. Sum = A + B
৪. Print Sum
৫. Stop

Flowchart



Key idea

Input
↓
Process
↓
Output

৯. Simple Problem Example: Even না Odd

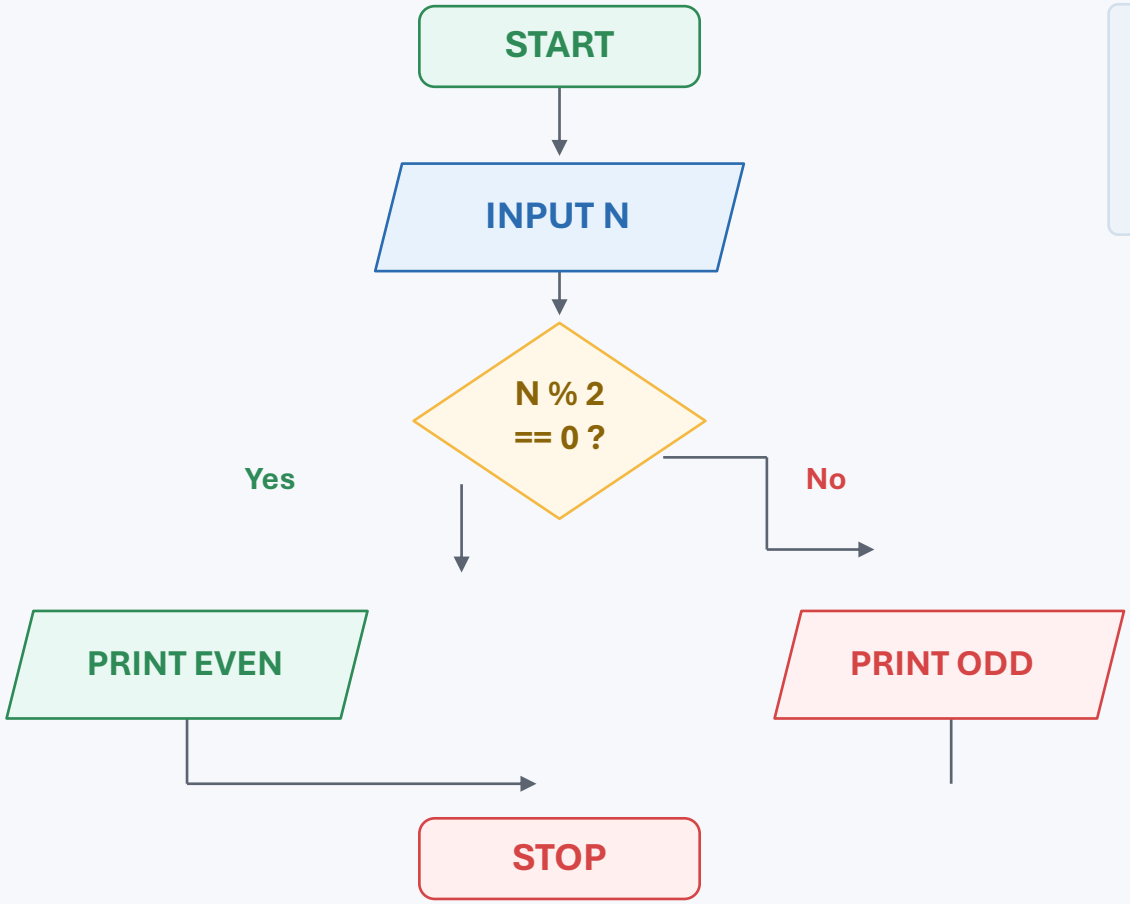
১.৬

Decision Symbol-এর ব্যবহার

Algorithm

১. Start
২. Input N
৩. যদি N % 2 == 0 হয়, তবে Print “Even”
৪. নাহলে Print “Odd”
৫. Stop

Flowchart



Decision
=
Condition
check

১০. Process of Program Planning

কোড লেখার আগের পরিকল্পনা সবচেয়ে গুরুত্বপূর্ণ

ভালো Program সাধারণত ভালো Planning দিয়েই শুরু হয়।
হয়।

Problem Definition → Analysis → Algorithm / Flowchart → Coding → Testing & Debugging → Documentation & Maintenance

ধাপগুলোর মানে

Problem বুঝো → Input/Output বিশ্লেষণ করো → Algorithm/Flowchart তৈরি করো → Code লিখো → Error ঠিক করো → পরে ব্যবহারের জন্য Documentation রাখো

কেন Planning জরুরি?

এতে ভুল কমে, Logic পরিষ্কার হয়, Coding সহজ হয়, এবং Program maintain করা সুবিধাজনক হয়।

১১. Recap + Viva Focus

Summary

আজকের লেকচার থেকে কী মনে রাখবে?

Recap

- Programming = কম্পিউটারকে কাজ করানোর নির্দেশনা লেখা
- Programming Language তিন ধরনের: Machine, Assembly, High Level
- Translator Program: Assembler, Compiler, Interpreter
- Algorithm = ধাপে ধাপে সমাধান
- Flowchart = Algorithm-এর চিত্ররূপ
- Program Planning = Problem থেকে Maintenance পর্যন্ত পূর্ণ পরিকল্পনা

Viva Questions

১. Programming কী?
২. Programming Language কী?
৩. Compiler ও Interpreter-এর পার্থক্য কী?
৪. Algorithm কী?
৫. Flowchart Symbol-এর কাজ কী?
৬. Program Planning কেন দরকার?

Next class suggestion: Variables, Data Types, Operators